SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

---

Code of Conduct:

I S L A K S H M I P R I Y A (1 9 2 5 2 4 2 8 4) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
S LAKSHMI PRIYA

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF.
(BL4 – Analyze)

Unnormalized Table (UNF)

Student_ID	Student_Name	Course_ID	Course_Name	Faculty	Faculty_Phone
101	Rahul	C101	DBMS	Ramesh	9876543210
101	Rahul	C102	Java	Suresh	9876543211
102	Priya	C101	DBMS	Ramesh	9876543210

Problems

- Data redundancy.
- Update anomaly.
- Insertion anomaly.
- Deletion anomaly.

1NF

Separate repeating groups.

Student

Student_ID	Student_Name
101	Rahul
102	Priya

Course

Course_ID	Course_Name	Faculty
C101	DBMS	Ramesh
C102	Java	Suresh

Enrollment

Student_ID	Course_ID
101	C101
101	C102
102	C101

2NF

Remove partial dependency.

Faculty

Faculty_ID	Faculty_Name	Phone
F101	Ramesh	9876543210
F102	Suresh	9876543211

Course

Course_ID	Course_Name	Faculty_ID
C101	DBMS	F101
C102	Java	F102

3NF

Final Tables

Student (Student_ID, Student_Name)
Faculty (Faculty_ID, Faculty_Name, Phone)
Course (Course_ID, Course_Name, Faculty_ID)
Enrollment (Student_ID, Course_ID)

Justification
Normalization removes redundancy and eliminates update, insertion, and deletion anomalies. 3NF improves consistency and maintains data integrity.

Q2. A company database contains employee and department details. Design SQL queries to retrieve employees working in multiple departments and justify your approach.
(BL6 – Create)

Tables

Employee

Emp_ID	Emp_Name

Department

Dept_ID	Dept_Name

Employee_Department

Emp_ID	Dept_ID

```
mysql> CREATE TABLE Department(
    -> Dept_ID VARCHAR(5) PRIMARY KEY,
    -> Dept_Name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Employee_Department(
    -> Emp_ID INT,
    -> Dept_ID VARCHAR(5),
    -> PRIMARY KEY(Emp_ID,Dept_ID),
    -> FOREIGN KEY(Emp_ID) REFERENCES Employee(Emp_ID),
    -> FOREIGN KEY(Dept_ID) REFERENCES Department(Dept_ID)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO Employee VALUES
    -> (101,'Rahul'),
    -> (102,'Priya'),
    -> (103,'Kiran');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Department VALUES
    -> ('D101','HR'),
    -> ('D102','IT'),
    -> ('D103','Finance');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO Department VALUES
    -> ('D101','HR'),
    -> ('D102','IT'),
    -> ('D103','Finance');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Employee_Department VALUES
    -> (101,'D101'),
    -> (101,'D102'),
    -> (102,'D102'),
    -> (103,'D103');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT E.Emp_ID,
    -> E.Emp_Name,
    -> COUNT(ED.Dept_ID) AS Total_Departments
    -> FROM Employee E
    -> JOIN Employee_Department ED
    -> ON E.Emp_ID=ED.Emp_ID
    -> GROUP BY E.Emp_ID,E.Emp_Name
    -> HAVING COUNT(ED.Dept_ID)>1;
+--------+----------+-------------------+
| Emp_ID | Emp_Name | Total_Departments |
+--------+----------+-------------------+
|    101 | Rahul    |                 2 |
+--------+----------+-------------------+
1 row in set (0.01 sec)
```

Justification
A JOIN combines employee and department information, while GROUP BY and HAVING identify employees assigned to more than one department efficiently.

Q3. Given a relation with multiple functional dependencies, determine the candidate keys and check BCNF. If not, decompose it.
(BL5 – Evaluate)

Relation

R(Student_ID, Course_ID, Faculty)

FDs

(Student_ID, Course_ID) → Faculty

Faculty → Course_ID

Candidate Key

(Student_ID, Course_ID)

BCNF Check

Faculty → Course_ID

Faculty is not a candidate key.

Hence,

Relation is NOT in BCNF.

BCNF Decomposition

R1(Faculty,Course_ID)

R2(Student_ID,Faculty)

```
CREATE TABLE FacultyCourse(
Faculty VARCHAR(30),
Course_ID VARCHAR(5),
PRIMARY KEY(Faculty)
);

CREATE TABLE StudentFaculty(
Student_ID INT,
Faculty VARCHAR(30),
PRIMARY KEY(Student_ID,Faculty)
);
```

```
mysql> INSERT INTO FacultyCourse VALUES
    -> ('Ramesh','C101'),
    -> ('Suresh','C102');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO StudentFaculty VALUES
    -> (101,'Ramesh'),
    -> (101,'Suresh'),
    -> (102,'Ramesh');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM StudentFaculty;
+------------+---------+
| Student_ID | Faculty |
+------------+---------+
|        101 | Ramesh  |
|        101 | Suresh  |
|        102 | Ramesh  |
+------------+---------+
3 rows in set (0.00 sec)

mysql>
```

Justification
BCNF removes dependencies where the determinant is not a candidate key, reducing redundancy and preventing update anomalies.

Q4. A banking system requires secure data access. Design views and constraints to ensure data security and integrity.

Create Tables

```
mysql> CREATE TABLE Account(
    -> Account_No INT PRIMARY KEY,
    -> Customer_Name VARCHAR(30),
    -> Balance DECIMAL(10,2) CHECK(Balance>=0),
    -> Branch VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Insert Data

```
mysql> INSERT INTO Account VALUES
    -> (1001,'Rahul',25000.00,'Chennai'),
    -> (1002,'Priya',40000.00,'Bangalore'),
    -> (1003,'Kiran',18000.00,'Hyderabad'),
    -> (1004,'Sneha',52000.00,'Chennai');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

View and constraints used

```
mysql> CREATE VIEW Customer_View AS
    -> SELECT Account_No,
    -> Customer_Name,
    -> Balance
    -> FROM Account;
Query OK, 0 rows affected (0.02 sec)

mysql> PRIMARY KEY(Account_No)
    ->
    -> CHECK(Balance>=0)
    ->
    -> NOT NULL(Customer_Name)
    ->
    -> UNIQUE(Account_No)
```

```
mysql> SELECT * FROM Account;
+------------+---------------+---------+-----------+
| Account_No | Customer_Name | Balance | Branch    |
+------------+---------------+---------+-----------+
|       1001 | Rahul         | 25000.00 | Chennai   |
|       1002 | Priya         | 40000.00 | Bangalore |
|       1003 | Kiran         | 18000.00 | Hyderabad |
|       1004 | Sneha         | 52000.00 | Chennai   |
+------------+---------------+---------+-----------+
4 rows in set (0.00 sec)
```

Justification

The view restricts users to only the required information, hiding sensitive columns such as Branch. The constraints (PRIMARY KEY, NOT NULL, and CHECK) ensure data integrity by preventing duplicate records, null values, and invalid account balances.

Q5. A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF.
(BL4 – Analyze)

Unnormalized Relation

Sales(Order_ID,
Customer_Name,
Customer_Address,
Product_ID,
Product_Name,
Quantity)

Problems

- Redundant customer data.
- Product details repeated.
- Update anomalies.

3NF Tables

Customer, Product , Orders

```
mysql>
mysql> CREATE TABLE Customer(
    -> Customer_ID INT PRIMARY KEY,
    -> Customer_Name VARCHAR(30),
    -> Customer_Address VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Product(
    -> Product_ID INT PRIMARY KEY,
    -> Product_Name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Orders(
    -> Order_ID INT PRIMARY KEY,
    -> Customer_ID INT,
    -> FOREIGN KEY(Customer_ID)
    -> REFERENCES Customer(Customer_ID)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Insert data

```
mysql> INSERT INTO Customer VALUES
    -> (101,'Rahul','Chennai'),
    -> (102,'Priya','Bangalore');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Product VALUES
    -> (201,'Laptop'),
    -> (202,'Mouse'),
    -> (203,'Keyboard');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Orders VALUES
    -> (1001,101),
    -> (1002,102);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> CREATE TABLE Order_Details(
    -> Order_ID INT,
    -> Product_ID INT,
    -> Quantity INT,
    -> PRIMARY KEY(Order_ID,Product_ID),
    -> FOREIGN KEY(Order_ID)
    -> REFERENCES Orders(Order_ID),
    -> FOREIGN KEY(Product_ID)
    -> REFERENCES Product(Product_ID)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Orders VALUES
    -> (1001,101),
    -> (1002,102);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

OUTPUT

```
mysql> SELECT O.Order_ID,
    ->        C.Customer_Name,
    ->        P.Product_Name,
    ->        OD.Quantity
    -> FROM Customer C
    -> JOIN Orders O
    -> ON C.Customer_ID = O.Customer_ID
    -> JOIN Order_Details OD
    -> ON O.Order_ID = OD.Order_ID
    -> JOIN Product P
    -> ON OD.Product_ID = P.Product_ID;
+----------+---------------+--------------+----------+
| Order_ID | Customer_Name | Product_Name | Quantity |
+----------+---------------+--------------+----------+
|     1001 | Rahul         | Laptop       |        1 |
|     1001 | Rahul         | Mouse        |        2 |
|     1002 | Priya         | Keyboard     |        1 |
+----------+---------------+--------------+----------+
3 rows in set (0.01 sec)
```

Justification

The sales database is normalized into separate Customer, Product, Orders, and Order_Details tables to eliminate redundancy and remove update, insertion, and deletion anomalies. The original sales information can be reconstructed efficiently using JOIN operations while maintaining data integrity.

Q6. A dataset of customers and orders. Write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

Create Tables customer, orders

```
mysql> CREATE TABLE Customer(
    -> Customer_ID INT PRIMARY KEY,
    -> Customer_Name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Orders(
    -> Order_ID INT PRIMARY KEY,
    -> Customer_ID INT,
    -> Order_Date DATE,
    -> FOREIGN KEY(Customer_ID)
    -> REFERENCES Customer(Customer_ID)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Insert data

```
mysql> INSERT INTO Orders VALUES
    -> (1001,101,'2026-01-10'),
    -> (1002,101,'2026-02-15'),
    -> (1003,102,'2026-03-20'),
    -> (1004,101,'2026-04-18'),
    -> (1005,103,'2026-05-25');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Join Query

```
mysql> SELECT C.Customer_Name,
    -> O.Order_ID,
    -> O.Order_Date
    -> FROM Customer C
    -> JOIN Orders O
    -> ON C.Customer_ID=O.Customer_ID;
+---------------+----------+------------+
| Customer_Name | Order_ID | Order_Date |
+---------------+----------+------------+
| Rahul         |     1001 | 2026-01-10 |
| Rahul         |     1002 | 2026-02-15 |
| Rahul         |     1004 | 2026-04-18 |
| Priya         |     1003 | 2026-03-20 |
| Kiran         |     1005 | 2026-05-25 |
+---------------+----------+------------+
5 rows in set (0.00 sec)
```

Output

```
mysql> SELECT Customer_Name
    -> FROM Customer
    -> WHERE Customer_ID IN
    -> (
    -> SELECT Customer_ID
    -> FROM Orders
    -> GROUP BY Customer_ID
    -> HAVING COUNT(*)>2
    -> );
+---------------+
| Customer_Name |
+---------------+
| Rahul         |
+---------------+
1 row in set (0.01 sec)
```

Justification

JOIN combines customer and order details, while the subquery identifies customers who have placed more than two orders. This efficiently retrieves meaningful business information.

Q7. A relation contains multivalued dependencies. Analyze the structure and decompose it into 4NF.
(BL4 – Analyze)

Relation

- Student(
- Student_ID,
- Hobby,
- Language

Sample Data

Student_ID	Hobby	Language
101	Cricket	English
101	Music	English
101	Cricket	Hindi
101	Music	Hindi

Multivalued Dependencies

Student_ID →→ Hobby

Student_ID →→ Language

The hobbies and languages are independent of each other, causing redundancy.

4NF Decomposition

Student Hobby , Student language

```
mysql> CREATE TABLE Student_Hobby(
    -> Student_ID INT,
    -> Hobby VARCHAR(30),
    -> PRIMARY KEY(Student_ID,Hobby)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Student_Language(
    -> Student_ID INT,
    -> Language VARCHAR(30),
    -> PRIMARY KEY(Student_ID,Language)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Insert data

```
mysql> INSERT INTO Student_Hobby VALUES
    -> (101,'Cricket'),
    -> (101,'Music');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student_Language VALUES
    -> (101,'English'),
    -> (101,'Hindi');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Output

```
mysql> SELECT * FROM Student_Hobby;
+------------+---------+
| Student_ID | Hobby   |
+------------+---------+
|        101 | Cricket |
|        101 | Music   |
+------------+---------+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Student_Language;
+------------+----------+
| Student_ID | Language |
+------------+----------+
|        101 | English  |
|        101 | Hindi    |
+------------+----------+
2 rows in set (0.00 sec)
```

Justification
4NF removes multivalued dependencies by separating independent attributes into different relations. This eliminates redundancy and prevents unnecessary duplication of data.

Q8. A database designer used improper constraints leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.
(BL5 – Evaluate)

Create Table

```
mysql> CREATE TABLE Employee(
    -> Emp_ID INT PRIMARY KEY,
    -> Emp_Name VARCHAR(30) NOT NULL,
    -> Email VARCHAR(50) UNIQUE,
    -> Salary DECIMAL(10,2) CHECK(Salary>0),
    -> Dept_ID VARCHAR(5)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Insert Valid Data

```
mysql>
mysql> INSERT INTO Employee VALUES
    -> (102,'Priya','priya@gmail.com',45000,'D102');
Query OK, 1 row affected (0.01 sec)
```

Display of the table

```
mysql> SELECT * FROM Employee;
+--------+----------+-----------------+----------+---------+
| Emp_ID | Emp_Name | Email           | Salary   | Dept_ID |
+--------+----------+-----------------+----------+---------+
|    101 | Rahul    | rahul@gmail.com | 35000.00 | D101    |
|    102 | Priya    | priya@gmail.com | 45000.00 | D102    |
+--------+----------+-----------------+----------+---------+
2 rows in set (0.00 sec)
```

Test Constraint

```
mysql> INSERT INTO Employee
    -> VALUES
    -> (103,'Kiran','rahul@gmail.com',-5000,'D103');
```

OUTPUT:

```
mysql> INSERT INTO Employee
    -> VALUES
    -> (103,'Kiran','rahul@gmail.com',-5000,'D103');
ERROR 3819 (HY000): Check constraint 'employee_chk_1' is violated.
mysql>
```

Integrity Constraints Used

- **PRIMARY KEY** → Prevents duplicate employee IDs.
- **NOT NULL** → Employee name cannot be empty.
- **UNIQUE** → Prevents duplicate email addresses.
- **CHECK** → Ensures salary is greater than zero.

Justification
Integrity constraints maintain data accuracy and consistency by preventing invalid or duplicate entries. They improve database reliability and reduce the chances of inconsistent data.

Q9. For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.
(BL6 – Create)

Given Relation

- Supply(
- Supplier_ID,
- Product_ID,
- Project_ID
-)

Sample Data

Supplier_ID	Product_ID	Project_ID
S1	P1	J1
S1	P2	J1
S2	P1	J2

Join Dependency

The relation contains a Join Dependency because suppliers, products, and projects can be stored independently and later joined.

Decomposition into 5NF

Create tables – Supplier product , supplier project , product project

```
mysql> CREATE TABLE Supplier_Product(
    -> Supplier_ID VARCHAR(5),
    -> Product_ID VARCHAR(5),
    -> PRIMARY KEY(Supplier_ID,Product_ID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Supplier_Project(
    -> Supplier_ID VARCHAR(5),
    -> Project_ID VARCHAR(5),
    -> PRIMARY KEY(Supplier_ID,Project_ID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Product_Project(
    -> Product_ID VARCHAR(5),
    -> Project_ID VARCHAR(5),
    -> PRIMARY KEY(Product_ID,Project_ID)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Insert data

```
mysql> INSERT INTO Supplier_Product VALUES
    -> ('S1','P1'),
    -> ('S1','P2'),
    -> ('S2','P1');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Supplier_Project VALUES
    -> ('S1','J1'),
    -> ('S2','J2');
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Product_Project VALUES
    -> ('P1','J1'),
    -> ('P2','J1'),
    -> ('P1','J2');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Display tables

```
mysql> SELECT * FROM Supplier_Product;
+-------------+------------+
| Supplier_ID | Product_ID |
+-------------+------------+
| S1          | P1         |
| S1          | P2         |
| S2          | P1         |
+-------------+------------+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Supplier_Project;
+-------------+------------+
| Supplier_ID | Project_ID |
+-------------+------------+
| S1          | J1         |
| S2          | J2         |
+-------------+------------+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Product_Project;
+------------+------------+
| Product_ID | Project_ID |
+------------+------------+
| P1         | J1         |
| P1         | J2         |
| P2         | J1         |
+------------+------------+
3 rows in set (0.00 sec)
```

OUTPUT

```
mysql> SELECT SP.Supplier_ID,
    -> SP.Product_ID,
    -> SJ.Project_ID
    -> FROM Supplier_Product SP
    -> JOIN Supplier_Project SJ
    -> ON SP.Supplier_ID=SJ.Supplier_ID
    -> JOIN Product_Project PJ
    -> ON SP.Product_ID=PJ.Product_ID
    -> AND SJ.Project_ID=PJ.Project_ID;
+-------------+------------+------------+
| Supplier_ID | Product_ID | Project_ID |
+-------------+------------+------------+
| S1          | P1         | J1         |
| S1          | P2         | J1         |
| S2          | P1         | J2         |
+-------------+------------+------------+
3 rows in set (0.00 sec)
```

Justification

5NF removes redundancy caused by join dependencies by decomposing the relation into smaller relations. The original relation can be reconstructed using JOIN operations without losing information.

Q10. A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.
(BL5 – Evaluate)

Create Tables

Student, department

```
mysql> CREATE TABLE Student(
    -> Student_ID INT PRIMARY KEY,
    -> Student_Name VARCHAR(30),
    -> Dept_ID VARCHAR(5),
    -> Marks INT
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Department(
    -> Dept_ID VARCHAR(5) PRIMARY KEY,
    -> Dept_Name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Insert Data

```
mysql> INSERT INTO Department VALUES
    -> ('D101','CSE'),
    -> ('D102','ECE');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Student VALUES
    -> (101,'Rahul','D101',85),
    -> (102,'Priya','D102',90),
    -> (103,'Kiran','D101',75);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Display of tables

```
mysql> SELECT Student_Name
    -> FROM Student
    -> WHERE Dept_ID=
    -> (
    -> SELECT Dept_ID
    -> FROM Department
    -> WHERE Dept_Name='CSE'
    -> );
+--------------+
| Student_Name |
+--------------+
| Rahul        |
| Kiran        |
+--------------+
2 rows in set (0.00 sec)

mysql> SELECT S.Student_Name,
    -> D.Dept_Name
    -> FROM Student S
    -> JOIN Department D
    -> ON S.Dept_ID=D.Dept_ID
    -> WHERE D.Dept_Name='CSE';
+--------------+-----------+
| Student_Name | Dept_Name |
+--------------+-----------+
| Rahul        | CSE       |
| Kiran        | CSE       |
+--------------+-----------+
2 rows in set (0.00 sec)
```

Creating index

```
mysql> CREATE INDEX idx_student
    -> ON Student(Student_ID);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

OUT[UT

```
mysql> SELECT * FROM Student;
+------------+--------------+---------+-------+
| Student_ID | Student_Name | Dept_ID | Marks |
+------------+--------------+---------+-------+
|        101 | Rahul        | D101    |    85 |
|        102 | Priya        | D102    |    90 |
|        103 | Kiran        | D101    |    75 |
+------------+--------------+---------+-------+
3 rows in set (0.00 sec)
```

Performance Comparison

Before Optimization	After Optimization
Subquery	JOIN
No Index	Index Created
Slower	Faster
Repeated Search	Optimized Search

Justification
Replacing subqueries with JOINs improves execution efficiency when retrieving related data. Creating indexes on frequently searched columns speeds up query execution, making the database more scalable and efficient.

OUTPUT